

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 1 – Experiments

Course Code:	DSA03o5	Course Title:	Natural Language Processing
CO Assessed:	CO3 – Precision	Assessment:	Assessment Tool 1 – Experiments
Weightage:	50% of CIA		
CO5 Statement:	Apply N-gram models, smoothing techniques, part-of-speech tagging systems and to evaluate effective syntactic analysis. (BL-3)		
Student Name:	N.Adi Narayana	Reg. No.:	192525346
Section / Batch:	AIML	Date:	08/08/2026

Code of Conduct

I N.Adi Narayana - 192525346 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____ Adi _____

Instructions

- Perform all experiments using Google Colab.
- Create a separate notebook (.ipynb) for the assessment.
- Ensure all code is executable without errors.
- Include appropriate comments explaining the logic used.
- Complete all three experiments in a single Google Colab notebook.
- Execute all cells and display the outputs for the given test cases.
- Save the notebook with the naming convention: RegNo_Name_CO3-AT1.ipynb
- Upload the notebook to your GitHub repository.
- Ensure the repository is public or accessible to the instructor.
- Submit the following:
 - GitHub Repository Link in GCR
 - Google Colab Share Link in GCR
 - Screenshots of uploaded coding and output files as printout.
- Duration: 30 minutes.

Section / Topic	Focus	Experiment Questions
Context-Free Grammars for English Syntax, Context-Free Rules and Trees, Sentence-Level Constructions, Parsing, Top-Down Parsing, Earley Parsing	Design a Python program to validate sentences using CFG production rules and construct parse tree representations for valid sentences.	Q1
Agreement, Feature Structures, Sub Categorization	Design a Python program to verify subject-verb agreement and validate grammatical constraints using feature structures and subcategorization rules.	Q2
Probabilistic Context-Free Grammars (PCFGs)	Design a Python program to parse sentences and compute sentence probabilities using probabilistic grammar production rules.	Q3

Annexure A – Experiments

Experiment 1: Context-Free Grammar Rule Validation and Parse Tree Construction

Experiment Description

Design a Python program to validate whether a sentence follows the given Context-Free Grammar (CFG) rules and generate its parse tree representation.

Grammar:

```
S → NP VP
NP → Det N
VP → V NP

Det → the | a
N → student | teacher | book
V → reads | likes
```

Task

1. Accept a sentence as input.
2. Verify whether it conforms to the CFG.
3. Construct and return the parse tree.
4. Return "Invalid Sentence" if parsing fails.

Function Prototype

```
def parse_cfg(sentence):
    pass
```

Test Cases

Input	Expected Output
the student reads a book	Valid Parse Tree
a teacher likes the book	Valid Parse Tree
student reads book	Invalid Sentence
the book likes a teacher	Valid Parse Tree
reads the student book	Invalid Sentence

Experiment 2: Agreement and Feature Structure Checking

Experiment Description

Design a Python program that verifies subject–verb agreement using feature structures containing Number and Person attributes.

Feature Rules

```
he → singular, third person
she → singular, third person
it → singular, third person
they → plural

runs → singular verb
writes → singular verb

run → plural verb
write → plural verb
```

Task

1. Accept Subject and Verb.

2. Compare feature structures.
3. Return True if agreement holds.
4. Return False otherwise.

Function Prototype

```
def check_agreement(subject, verb):
    pass
```

Test Cases

Subject Verb Expected

he	runs	True
he	run	False
they	run	True
they	writes	False
she	writes	True

Experiment 3: Probabilistic Context-Free Grammar (PCFG) Parsing

Experiment Description

Implement a Python program to compute the probability of a sentence using the given PCFG.

Grammar

$S \rightarrow NP VP$	[1.0]
$NP \rightarrow \text{John}$	[0.6]
$NP \rightarrow \text{Mary}$	[0.4]
$VP \rightarrow \text{runs}$	[0.5]
$VP \rightarrow \text{walks}$	[0.5]

Probability Formula

$$\begin{aligned}
 &P(\text{Sentence}) \\
 &= \\
 &P(S \rightarrow NP VP) \\
 &\times P(NP) \\
 &\times P(VP)
 \end{aligned}$$

Task

1. Accept a two-word sentence.
2. Parse using the PCFG.
3. Compute sentence probability.
4. Return 0 if sentence cannot be generated.

Function Prototype

```
def sentence_probability(sentence):
    pass
```

Test Cases

Input Sentence Expected Probability

John runs	0.30
John walks	0.30
Mary runs	0.20
Mary walks	0.20
Peter runs	0.00

Experiment 1: CFG Rule Validation and Parse Tree Construction

Aim

To design and implement a Python program to validate whether a given sentence conforms to the specified Context-Free Grammar (CFG) rules and construct its corresponding parse tree.

Algorithm

1. Start the program.
2. Accept a sentence from the user.
3. Split the sentence into individual words.
4. Define the CFG rules:
 - $S \rightarrow NP VP$
 - $NP \rightarrow Det N$
 - $VP \rightarrow V NP$
 - $Det \rightarrow the \mid a$
 - $N \rightarrow student \mid teacher \mid book$
 - $V \rightarrow reads \mid likes$
5. Check whether the sentence contains exactly five words.
6. Verify the first noun phrase using the pattern Det N.
7. Verify the verb phrase using the pattern V NP.
8. If all the words satisfy the grammar rules, construct the parse tree.
9. Otherwise, return "Invalid Sentence".
10. Stop the program.

Program

```
def parse_cfg(sentence):  
    words = sentence.split()  
  
    determiners = ["the", "a"]  
    nouns = ["student", "teacher", "book"]  
    verbs = ["reads", "likes"]  
  
    if len(words) != 5:  
        return "Invalid Sentence"  
  
    det1, noun1, verb, det2, noun2 = words  
  
    if (det1 in determiners and  
        noun1 in nouns and  
        verb in verbs and  
        det2 in determiners and
```

noun2 in nouns):

```
parse_tree = {  
    "S": {  
        "NP": {  
            "Det": det1,  
            "N": noun1  
        },  
        "VP": {  
            "V": verb,  
            "NP": {  
                "Det": det2,  
                "N": noun2  
            }  
        }  
    }  
}
```

return parse_tree

return "Invalid Sentence"

sentence = input("Enter a sentence: ")

result = parse_cfg(sentence)

print("Result:")

print(result)

output:

Programiz

Python Online Compiler

Programiz PRO >

main.py

Run

Clear

```

4 words = sentence.split()
5
6 # Grammar Rules
7 determiners = ["the", "a"]
8 nouns = ["student", "teacher", "book"]
9 verbs = ["reads", "likes"]
10
11 # Sentence should contain exactly 5 words
12 if len(words) != 5:
13     return "Invalid Sentence"
14
15 det1, noun1, verb, det2, noun2 = words
16
17 if (det1 in determiners and
18     noun1 in nouns and
19     verb in verbs and
20     det2 in determiners and
21     noun2 in nouns):
22
23     parse_tree = {
24         "S": {
25             "NP": {
26                 "Det": det1,
```

Output

Clear

```

Sentence: the student reads a book
{'S': {'NP': {'Det': 'the', 'N': 'student'}, 'VP': {'V': 'reads', 'NP': {'Det': 'a', 'N': 'book'}}}}

Sentence: a teacher likes the book
{'S': {'NP': {'Det': 'a', 'N': 'teacher'}, 'VP': {'V': 'likes', 'NP': {'Det': 'the', 'N': 'book'}}}}

Sentence: student reads book
Invalid Sentence

Sentence: the book likes a teacher
{'S': {'NP': {'Det': 'the', 'N': 'book'}, 'VP': {'V': 'likes', 'NP': {'Det': 'a', 'N': 'teacher'}}}}

Sentence: reads the student book
Invalid Sentence

=== Code Execution Successful ===
```

Experiment 2: Agreement and Feature Structure Checking

Aim

To design and implement a Python program to verify subject–verb agreement using feature structures containing Number and Person attributes.

Algorithm

1. Start the program.
2. Accept the subject and verb as input.
3. Define the feature structure for subjects:
 - he → singular, third person
 - she → singular, third person
 - it → singular, third person
 - they → plural
4. Define the corresponding features for verbs:
 - runs → singular
 - writes → singular
 - run → plural
 - write → plural
5. Check whether the given subject and verb exist in the feature structures.
6. Compare the number features of the subject and verb.
7. If the features match, return True.
8. Otherwise, return False.
9. Stop the program.

Program

```
def check_agreement(subject, verb):
```

```
subject_features = {  
    "he": "singular",  
    "she": "singular",  
    "it": "singular",  
    "they": "plural"  
}
```

```
verb_features = {  
    "runs": "singular",  
    "writes": "singular",  
    "run": "plural",  
    "write": "plural"  
}
```

```
if subject not in subject_features:  
    return False
```

```
if verb not in verb_features:  
    return False
```

```
return subject_features[subject] == verb_features[verb]
```

```
subject = input("Enter subject: ")  
verb = input("Enter verb: ")
```

```
result = check_agreement(subject, verb)
```

```
print("Agreement:", result)
```

Sample Input

Enter subject: he

Enter verb: runs

Sample Output

Agreement: True

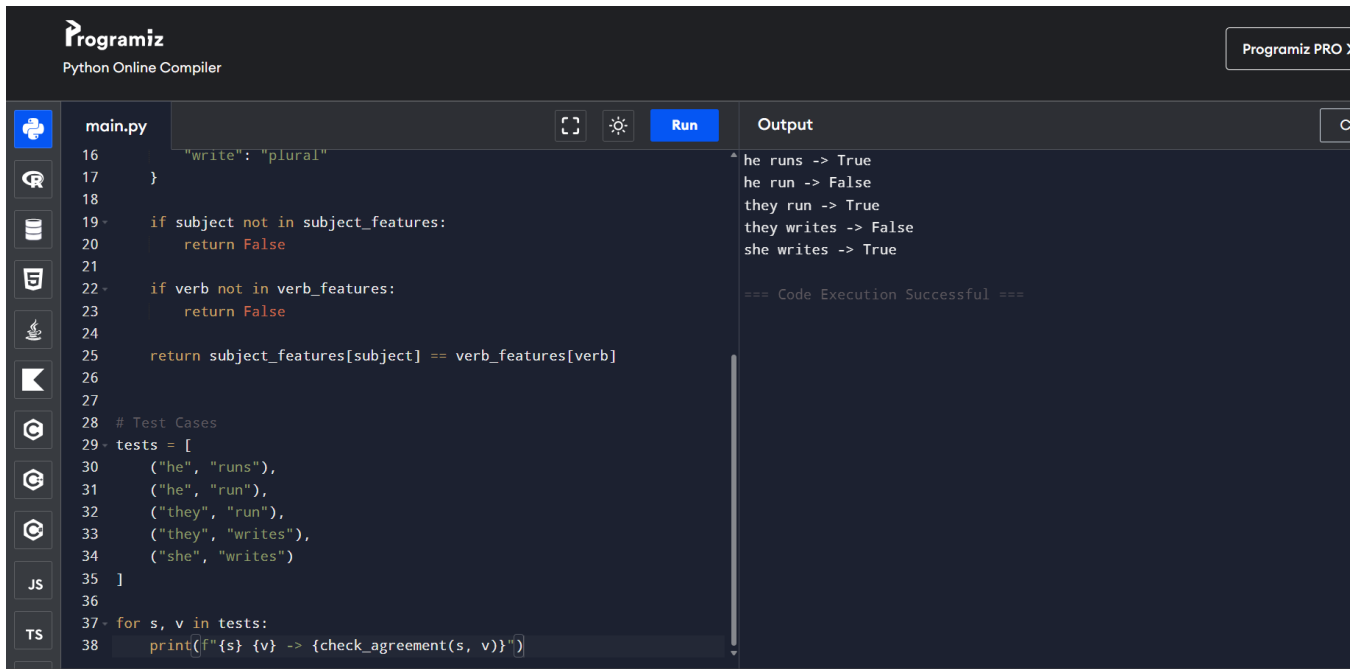
Another Sample Input

Enter subject: he

Enter verb: run

Output

Agreement: False



The screenshot shows the Programiz Python Online Compiler interface. The editor contains a Python script named `main.py` with the following code:

```
16     "write": "plural"
17 }
18
19 if subject not in subject_features:
20     return False
21
22 if verb not in verb_features:
23     return False
24
25 return subject_features[subject] == verb_features[verb]
26
27
28 # Test Cases
29 tests = [
30     ("he", "runs"),
31     ("he", "run"),
32     ("they", "run"),
33     ("they", "writes"),
34     ("she", "writes")
35 ]
36
37 for s, v in tests:
38     print(f'{s} {v} -> {check_agreement(s, v)}')
```

The output window on the right shows the results of the script execution:

```
he runs -> True
he run -> False
they run -> True
they writes -> False
she writes -> True

=== Code Execution Successful ===
```

Experiment 3: Probabilistic Context-Free Grammar (PCFG) Parsing

Aim

To design and implement a Python program to calculate the probability of a sentence using a Probabilistic Context-Free Grammar (PCFG).

Algorithm

1. Start the program.
2. Accept a two-word sentence from the user.
3. Define the PCFG probabilities:
 - $S \rightarrow NP VP$ [1.0]
 - $NP \rightarrow \text{John}$ [0.6]
 - $NP \rightarrow \text{Mary}$ [0.4]
 - $VP \rightarrow \text{runs}$ [0.5]
 - $VP \rightarrow \text{walks}$ [0.5]
4. Split the sentence into two words.
5. Check whether the first word can be generated by the NP rule.
6. Check whether the second word can be generated by the VP rule.

7. If either word is not present in the grammar, return probability 0.

8. Otherwise, calculate:

$$P(\text{Sentence}) = P(S \rightarrow NP VP) \times P(NP) \times P(VP)$$

9. Return the calculated probability.

10. Stop the program.

Program

```
def sentence_probability(sentence):
```

```
    noun_probabilities = {  
        "John": 0.6,  
        "Mary": 0.4  
    }
```

```
    verb_probabilities = {  
        "runs": 0.5,  
        "walks": 0.5  
    }
```

```
    words = sentence.split()
```

```
    if len(words) != 2:  
        return 0.0
```

```
    noun, verb = words
```

```
    if noun not in noun_probabilities:  
        return 0.0
```

```
    if verb not in verb_probabilities:  
        return 0.0
```

```
    probability = (  
        1.0 *  
        noun_probabilities[noun] *  
        verb_probabilities[verb]  
    )
```

```
    return probability
```

```
sentence = input("Enter a sentence: ")
```

```
probability = sentence_probability(sentence)
```

```
print("Sentence Probability:", probability)
```

Sample Input

Enter a sentence: John runs

Sample Output

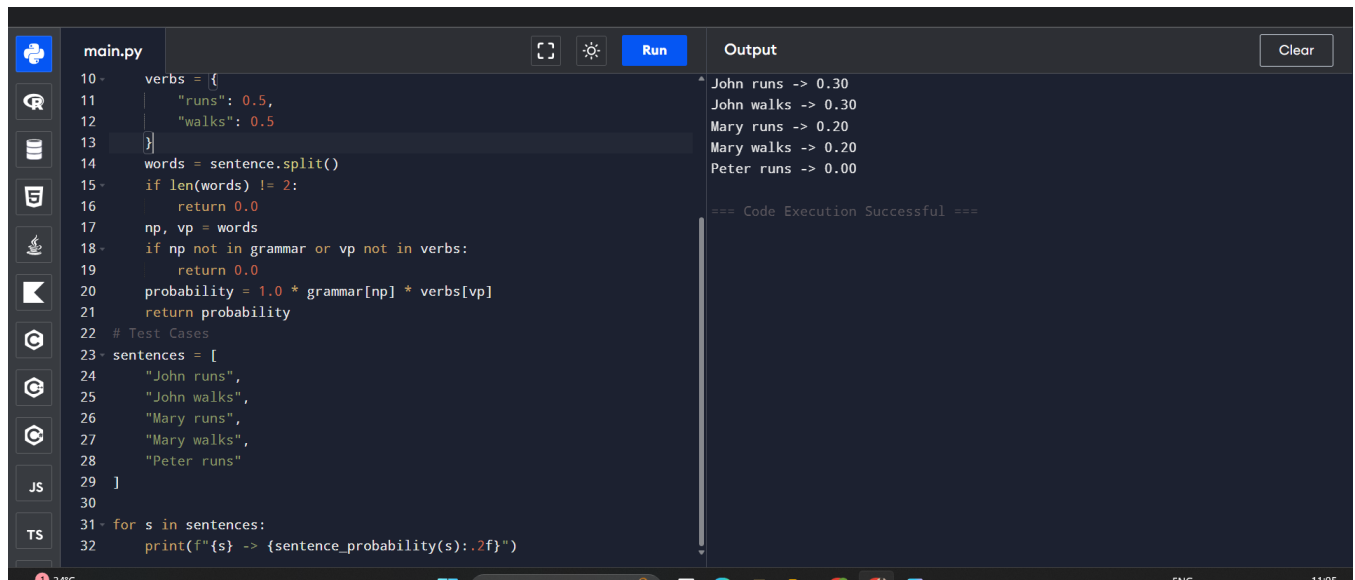
Sentence Probability: 0.3

Another Sample Input

Enter a sentence: Mary walks

Output

Sentence Probability: 0.2



The screenshot shows a code editor with a file named `main.py`. The code defines a function `sentence_probability` that calculates the probability of a sentence based on a grammar and a list of verbs. The grammar is a dictionary where keys are words and values are probabilities. The function splits the sentence into words and checks if the first word is in the grammar and the second word is in the list of verbs. If both conditions are met, it returns the product of the grammar value for the first word and the value for the verb. Otherwise, it returns 0.0. The code also includes test cases for the sentences "John runs", "John walks", "Mary runs", "Mary walks", and "Peter runs". The output of the code execution is shown in the right panel, displaying the probability for each sentence: "John runs" (0.30), "John walks" (0.30), "Mary runs" (0.20), "Mary walks" (0.20), and "Peter runs" (0.00). The code execution was successful.

```
10 verbs = {
11     "runs": 0.5,
12     "walks": 0.5
13 }
14 words = sentence.split()
15 if len(words) != 2:
16     return 0.0
17 np, vp = words
18 if np not in grammar or vp not in verbs:
19     return 0.0
20 probability = 1.0 * grammar[np] * verbs[vp]
21 return probability
22 # Test Cases
23 sentences = [
24     "John runs",
25     "John walks",
26     "Mary runs",
27     "Mary walks",
28     "Peter runs"
29 ]
30
31 for s in sentences:
32     print(f"{s} -> {sentence_probability(s):.2f}")
```

Output

```
John runs -> 0.30
John walks -> 0.30
Mary runs -> 0.20
Mary walks -> 0.20
Peter runs -> 0.00

=== Code Execution Successful ===
```

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Q. No. / Criteria Level	Understanding of Concepts	Experimental Design	Use of Tools	Analysis of Results	Documentation	Sub. Total	Total %
1							
2							
3							

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____